01 / OLAY YERİ

Bir dokunuş. İki gerçek.

Kullanıcının gördüğü sonuç ile sunucudaki işlem birbirinden ayrılıyor.



Bir kullanıcı niyeti

`POST /package-changes`

Kabul yanıtı cihaza ulaşmadı

× Bağlantı koptu

Kullanıcı yeniden gönderirse ikinci bir talep oluşabilir mi?

BACKEND OLAY KAYDI

```
İstek geldi
Uygunluk kontrolü geçti
Talep kaydı oluştu
```

Talep kabul edildi

KABUL, UYGULANMA ANLAMINA GELMEZ.

Ürünün bu durumda ne yapması gerektiğini nerede kararlaştırdık?

İlk kez kullanıyorum →

Temsili ürün · örnek kurallar

02 / EKSİK BAĞLANTI

Karar nerede kayboldu?

Bir talep cümlesi, kullanıcıya verilecek bütün sözleri tanımlamaz.

Ürün talebi

“Kullanıcı paketini değiştirebilsin.”

Niyet açık. Kritik durumlar henüz açık değil.

İncelenmiş davranış sözleşmesi

Koşul, beklenen sonuç ve açık kararlar birlikte görünür.

Mobil

Genel hata gösterir.

Backend

Talebi kabul eder.

QA

Yalnız başarılı gönderimi sınar.

Eksik karar, kodda ve testte farklı varsayımlara dönüşebilir.

Temsili vaka

03 / OPENSPEC'İN ZİHİNSEL MODELİ

Değişikliğin çalışma dosyası

OpenSpec, AI kod asistanları için gereksinimleri ve değişiklik planını sürümlenen dosyalarda tutar.

BUGÜN / DAVRANIŞ REFERANSI

openspec/specs/package-change/spec.md

Requirement: Gönderim sonucu
Kabul yanıtı geldiğinde talep durumunu göster.

BU İŞ / ÖNERİLEN DEĞİŞİKLİK

openspec/changes/recover-package-change-result/

proposal.md
specs/package-change/spec.md
design.md
tasks.md

MODIFIED Requirements
Sonuç bilinmiyorsa belirsizliği ve devam yolunu göster.

İŞ TAMAMLANINCA

openspec/specs/…/spec.md

Requirement: Gönderim sonucu
Doğrulanmış sonucu göster.
Belirsiz sonuçta durum sorgula.

changes/archive/<tarih>-recover-package-change-result/

Neden ve nasıl değiştiği korunur.

Spec ürünün davranış referansıdır. Güncelliğini ekip korur.

04 / REPO'DAN BAĞLAM

Bir akışın kanıt izi

Mobil, API ve backend aynı kullanıcı eylemi üzerinden incelenir.

mobile / PackageChangeViewModel
Temsili kod

```
submit(packageId)
  await api.change(packageId)
catch timeout
  showGenericError()
```

POST
/package-changes

backend / PackageChangeService
Temsili kod

```
checkEligibility(user)
saveAcceptedRequest()
return accepted

test: acceptsValidRequest
```

Kaynak ve sürüm	Dosya, sembol ve incelenen branch/commit kaydedilir.
Gözlem	Mobil timeout'ta hata gösterir. Backend daha önce kabul etmiş olabilir.
Açık soru	Canlı flag, eski istemci ve dış servis davranışı henüz doğrulanmadı.

Repo erişimi ve sürüm bilgisi gerekir. Repo adını yazmak erişim sağlamaz.

Kanıt kaydı: önerilen ekip pratiği

05 / PM'İN SORUMLULUĞU

Kanıt ve ürün kararı

Kod mevcut uygulamayı gösterir. Beklenen davranışın sahibi ürün ekibidir.

MOBİLDE GÖZLENDİ

Timeout olduğunda genel hata.

BACKEND'DE MÜMKÜN

Talep kaydı zaten oluşmuş olabilir.

?

Beklenen kullanıcı davranışı

~~*"Timeout olursa başarısız say."*~~

AI'ın temsili önerisi

"Sonuç doğrulanana kadar belirsizliği göster."

Gözlendi Bilinmiyor Kararlaştırılacak

AI çelişkiyi bulabilir. Bir davranışı ürün sözüne dönüştüren karar ekibe aittir.

Temsili ürün kararı

06 / PROPOSAL

Bir değişiklik, tek niyet

Dar kapsam, mobil ve backend'i kapsayan uçtan uca bir sonuç üretebilir.

`changes/recover-package-change-result/proposal.md`

Neden

Yanıt kaybolduğunda kullanıcı işlemin başarısız olduğunu sanıyor.

Değişecek davranış

Doğrulanamayan sonucu açık gösterme, aynı talebi güvenle toparlama ve mevcut durumu sorgulama.

Etkilenen alanlar

Mobil durum gösterimi, backend talep sözleşmesi ve ilgili testler.

Kapsamın dışında

- Fiyat ve faturalama
- Kampanya önerileri
- Geçmiş talepler ekranı

Uygunluk ve geçerlilik politikası bu işte değişmiyor.

07 / VARSAYILAN SPEC-DRİVEN YAPISI

Dört belge, tek karar izi

Aynı karar, amaçtan davranışa ve uygulama işine kadar tutarlı kalır.

`proposal.md`

Neden?

Belirsiz sonuçta güvenli devam.

PM niyet ve sınırı inceler.

`specs/…/spec.md`

Ne olmalı?

Aynı niyetin tekrar gönderimi ikinci bir talep oluşturmaz.

Gözlemlenebilir davranış.

`design.md`

Nasıl?

`operationKey` gönderimden önce oluşur.

`Teknik çözüm tercihi`

`tasks.md`

Hangi işler?

Aynı anahtarı koru.
Tekrar gönderimi uygula.
Eşzamanlı isteği test et.

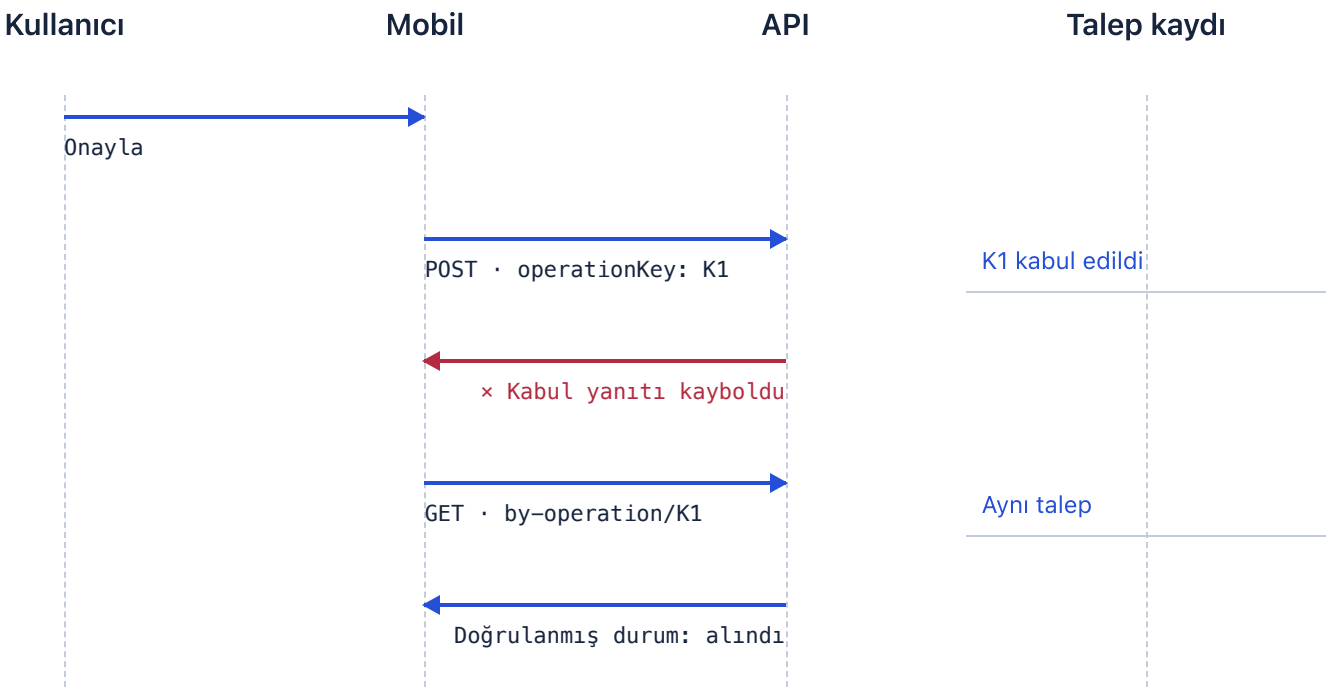
Ürün sözü: tek talep. Teknik karar: anahtar. Kanıt: tekrar ve eşzamanlılık testi.

PM, mühendislik ve QA farklı parçaları birlikte değerlendirir. Bu rol dağılımı ekip önerisidir.

Örnek teknik yaklaşım

İki repo arasındaki sözleşme

Gönderimden önce oluşan anahtar, yanıt kaybolsa da aynı niyeti izlemeyi sağlar.



Örnek API tasarımı. Yeniden gönderimde de aynı K1 kullanılır.

MOBİL BİLGİ

Paket değişikliği

Yeni paket

Talebiniz alındı

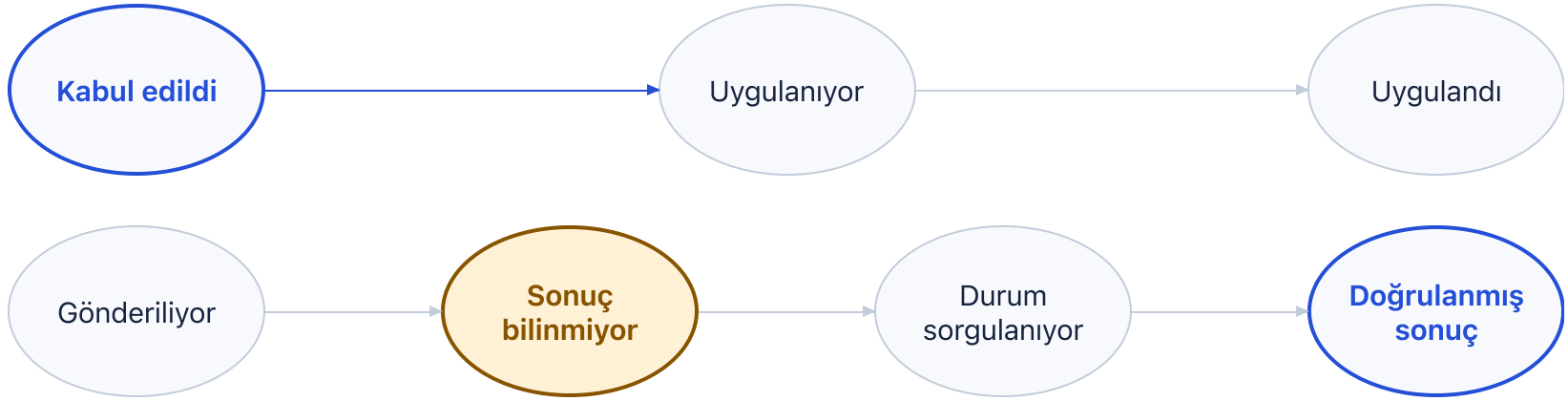
"Alındı", "aktif" demek değildir.

İkinci işlem yaratmamak ürün gereksinimidir. Anahtar ve atomiklik mühendislik tasarımıdır.

09 / DURUM MODELİ

İşlem durumu ve bilinen durum

Bağlantı kesildiğinde sunucu çalışmaya devam edebilir.



> Ekrana dönüşte sorgulama, istemcinin bilgisini sunucuyla yeniden eşleştirir.

"Sonuç bilinmiyor" bir istemci bilgi durumudur. Örnek sunucu akışının hata dalları ayrıca tanımlanır.

Sadeleştirilmiş temsili durum modeli

Bir söz, bir sınanabilir senaryo

Gereksinim davranış sözünü, senaryo o sözü sınayacak somut durumu tanımlar.

specs/package-change/spec.md

```
### Requirement: Belirsiz sonucun gösterimi
Sistem, sonuç doğrulanamadığında
“Sonuç kontrol ediliyor” göstermelidir (MUST).

#### Scenario: Kabul yanıtı cihaza ulaşmaz
- GIVEN paket değişikliği talebi gönderildi
- WHEN yanıt alınamadı ve sonuç doğrulanamadı
- THEN kullanıcı belirsiz sonuç durumunu görür
```

GIVEN — Kullanıcı talebi gönderdi.
Başlangıç koşulu

WHEN — Yanıt cihaza ulaşmadı.
Tetikleyen olay

THEN — Kesin hata yerine doğru bilgi.

> Sonuç kontrol ediliyor

“Hataları düzgün yönetir” yerine, dışarıdan gözlenebilen bir sonuç tanımlanır.

Mevcut spec'e işlenen fark

Bu vakada mevcut "Gönderim sonucu" gereksiniminin yeni tam hâli yazılır.

ÖNCE / ANA SPEC

```
### Requirement: Gönderim sonucu
~~Yanıt yoksa hata göster.~~

#### Scenario: Timeout
- WHEN yanıt alınamaz
- THEN hata gösterilir

### Requirement: Uygunluk
Mevcut uygunluk kuralı.
```

DELTA / CHANGE İÇİNDE

```
## MODIFIED Requirements

### Requirement: Gönderim sonucu
Sonuç doğrulanamadığında
belirsizliği göster (MUST).

#### Scenario: Timeout
- WHEN yanıt alınamaz
- AND sonuç doğrulanamaz
- THEN "Sonuç kontrol
  ediliyor" gösterilir
```

SONRA / ANA SPEC

```
### Requirement: Gönderim sonucu
Belirsiz sonucu doğru göster.

#### Scenario: Timeout
- WHEN sonuç doğrulanamaz
- THEN "Sonuç kontrol
  ediliyor" gösterilir

### Requirement: Uygunluk
Mevcut uygunluk kuralı.
```

MODIFIED, tam yeni gereksinimi ve senaryolarını içerir. Yeni davranış ADDED, kaldırma REMOVED ile tarif edilir.

Kısaltılmış temsili diff

12 / İNCELEME EGZERSİZİ

AI taslağındaki üç kusur

Sorunlu satıra dokunun. Gerekçeyi gördükten sonra taslağı düzeltin.

01	~~Talep 24 saat içinde uygulanır.~~	Kaynağı olmayan iş kuralı. Geçerlilik politikasını domain sahibine sorun.
02	Tüm hatalarda yeniden gönder.	Sonucu bilinmeyen işlem de başarısız sayılıyor. Önce durumu doğrulayın.
03	Geçmiş talepler ekranı ekle.	Kapsam sızıntısı. Bu iş güvenli toparlanma ile sınırlı.

Temiz taslak: doğrulanmış durumu göster. Belirsiz sonuçta sorgula. Aynı niyeti tekrar işlem yaratmadan toparla.

İnceleme sırası: proposal, spec'ler, tasks. Teknik yaklaşımı gerektiğinde mühendislikle değerlendirin.

Öğrendikçe değişen çalışma döngüsü

Plan yeni bilgiyle değişebilir. Uygulama, güncel plana yeniden uyarlanır.

`/opsx:explore`

Bağlamı ve seçenekleri araştır

İsteğe bağlı başlangıç

`/opsx:propose`

Değişiklik belgelerini oluştur

`changes/recover-package-change-result/`

`proposal · specs · design · tasks`

İncelenen plan, uygulamanın bağlamıdır.

> Yeni bilgi geldi.
> /opsx:update ile planı düzelt.
> /opsx:apply ile kodu uyumla.

`/opsx:apply`

Görevleri uygula ve değerlendir

`/opsx:sync · /opsx:archive`

Deltaları işle, geçmişi koru

Sync ayrıca kullanılabilir.

TERMINAL / CLI

`openspec init` · `openspec update`

AI SOHBETİ / WORKFLOW

`/opsx:propose` · Codex: `$openspec-propose`

/opsx:update plan belgelerini değiştirir. Kod değişikliği için apply gerekir.

Archive gerektiğinde sync teklif eder.

“Bitti” demenin kanıtı

AI’ın “tüm görevler tamamlandı” raporu, kabul senaryolarının geçtiğini tek başına kanıtlamaz.

`openspec validate <change>`

Format ve yapı kontrolü

Davranışın doğru çalıştığını göstermez.

`/opsx:verify`

Uygulama ve belge karşılaştırması

Expanded workflow. Zorunlu arşiv kapısı değil.

Test, demo ve inceleme

İlgili senaryonun gerçek sonuç kaydı gerekir.

SENARYO	BEKLENEN KANIT	ÖRNEK KAYIT
Yanıt kaybolur	Belirsiz durum görünür	Kanıt istenecek
Aynı niyet tekrar gönderilir	Tek talep oluşur	Kanıt istenecek
Uygulamaya geri dönülür	Durum yeniden sorgulanır	**Çalıştırılmadı**
Kesin ret gelir	Gerekçe ve devam yolu görünür	Kanıt istenecek

Temsili kabul tablosu. Bu sunumda ürün testi çalıştırılmadı.

Archive değişikliği arşivler. Yayına çıkışı veya ürün kabulünü gerçekleştirmez.

İki repo, bir ortak sözleşme

Davranış bir yerde sahiplenilir. Her uygulama kendi uygulama işlerini taşır.

STORES BETA / AYRI GIT PLAN REPO'SU

Paket değişikliği sözleşmesi

`Requirement: aynı niyet ikinci talep oluşturmaz`

Mobil repo

Durum gösterimi ve toparlanma.
Anahtarı aynı niyet için koruma.

Mobil testleri ve incelemesi

Backend repo

Aynı niyeti tek talebe eşleme.
Durumu güvenilir sorgulama.

Servis testleri ve incelemesi

Sözleşme revizyonu · mobil sürüm · backend sürüm · eski istemci uyumluluğu birlikte izlenir.

Stores ayrı repo'da planlama sağlar ve beta'dır. Kod erişimi, Git paylaşımı ve sürüm eşlemesi ayrıca yönetilir.

Artık siz yönetin

Neyi biliyoruz? Neye karar verdik? Neyle doğruladık?

Ürün niyeti, incelenebilir sözleşme.

Değişiklik, kalıcı ürün hafızası.

- Geçerlilik politikasını domain sahibine sorun.
- Uygulamaya geri dönünce durumun toparlanmasını ekleyin.
- Aynı anahtarla eşzamanlı tekrar testi sonucunu isteyin.

Sipariş iptali akışını mobil ve backend kaynaklarından incele. Gözlemleri kaynaklarıyla, varsayımları ve açık ürün kararlarını ayrı yaz. Amaç ve kapsam dışını bizimle netleştir. Domain kurallarını kaynak olmadan varsayma. Kararlar netleşince dar kapsamlı proposal ve senaryo taslağı oluştur.

İlk pilot: bir gerçek akış, PM, geliştirici ve QA. Sonraki işte yöntemi bağımsız uygulayın.

UYGULAMA 01 / 10 · BİLGİSAYARDAN İLK İŞE

Bir kere kur. Her ürüne ayrı bağlam ver.

CLI kurulumu, skill kapsamı ve ürün hafızası üç ayrı katmandır.

01	**Kullanıcı hesabı** CLI + kişisel araç tercihleri	`npm prefix -g` → Kurulum dizini `openspec config path` → Kişisel ayar dosyası
02	**Asistanın çalışma yöntemi** OpenSpec workflow skill'leri	`<repo>/.agents/skills/` veya `~/.agents/skills/`
03	**Ürün / domain** Kurallar + kaynaklar + değişiklikler	`<root>/openspec/config.yaml` `docs/context/ + specs/`

Öneri: ortak yöntem kullanıcıya; domain bilgisi ilgili projenin sürümlenen bağlamına.

CLI gerçekten user altında nasıl kurulur?

Önce runtime ve hedefi kontrol et. Kurulumun çalıştığını sürüm çıktısıyla doğrula.

01 / Mevcut Node düzeni

```
# macOS / Linux · terminal
node --version           # >= 20.19.0
npm --version
npm prefix -g            # mevcut hedefi gör

# Mevcut Node yöneticin varsa onun yolunu kullan:
npm install -g @fission-ai/openspec@latest
openspec --version
```

02 / Açıkça home hedefi

```
# Alternatif: açıkça home altında npm kurulumu
npm install -g --prefix "$HOME/.local" @fission-
ai/openspec@latest
export PATH="$HOME/.local/bin:$PATH"
command -v openspec
openspec --version
```

Bu export yalnız mevcut shell'e etki eder. Kalıcılık için PATH satırını kullandığınız shell'in başlangıç dosyasına ekleyin.

–g hedefi prefix belirler. nvm/fnm kullanıyorsanız CLI aktif Node sürümüne bağlıdır.

“Her projemde skill görünsün” diyorsan…

Varsayılan yol proje kurulumu. Kullanıcı genelinde dağıtım ayrıca yönetilir.

NORMAL PROJE KURULUMU

```
product-repo/
├─ .agents/skills/openspec-*/
└─ openspec/
```

Repo’da

```
openspec init --tools codex
```

. Ekibin yöntemi repo ile taşınır.

İSTEĞE BAĞLI USER KAPSAMI

```
~/.agents/skills/openspec-*/
↳ kişisel toolkit’ten symlink
```

Skill bütün projelerde keşfedilir. Ürün bağlamı hâlâ ilgili root’tadır.

Kişisel toolkit · ayrı ve güncellenebilir

```
~/.local/share/openspec-toolkit/
  .agents/skills/openspec-*/SKILL.md

# Sonra bu toolkit içinde:
openspec update
```

Bu, Codex’in user skill desteğinden yararlanan önerilen manuel düzenlemedir. OpenSpec’in `init --global` komutu değildir.

Aynı isimli user ve repo skill’leri birleşmez; ikisi seçicide görünebilir. Bir bakım yolu seçin.

UYGULAMA 04 / 10 · BİLGİSAYARDAN İLK İŞE

Mobil + backend: komut nerede çalışacak?

Repo URL'si, yerel dosya erişimi ve OpenSpec root'u birbirinden farklıdır.

A / TEK REPO

Repo root'u

```
product/
  mobile/
  backend/
  openspec/
```

Bir monorepo; ortak root ve ilgili domain spec'leri.

B / İLK ÇOKLU REPO PİLOTU

Planlama root'u

```
work/
  product-plans/ ← PM burada
  mobile-app/
  backend/
```

Plan burada; iki kod checkout'u asistana ayrıca erişilebilir.

C / EKİP ÖLÇEĞİ · BETA

Store + component

```
team-plans/ ← ortak spec
mobile/     ← yerel change
backend/    ← yerel change
```

Ortak sözleşme referanslanır. Component uygulaması kendi root'unda.

Terminal: seçilen root'ta `openspec init --tools codex`. Sohbet: root + kod yolları + erişim kapsamını açık belirt.

En güçlü bağlam: küçük giriş, derin kanıt.

AI'a okuma rotası ver; ilgili dosyaları gerçekten okuduğunu kaynaklarla göstermesini iste.

01	İşin niyeti	Kullanıcı · sorun · beklenen sonuç · kapsam dışı
02	Domain paketi	Sözlük · kurallar · akış · API · karar sahipleri
03	İlgili kaynak dilimi	Mobil ekran → API → handler → domain → veri → test
04	Çalışma gerçekliği	App release · deploy · ortam · flag · dış servis
05	Değişiklik ve kabul	İlgili spec · delta · kritik senaryolar · ölçüm

Güç = ilgililik + kaynak + sürüm + açık karar. Bütün repo'yu sohbet içine yapıştırmak bu dört şeyi sağlamaz.

UYGULAMA 06 / 10 · BİLGİSAYARDAN İLK İŞE

Repo’dan bağlam çıkar. Kararı uydurma.

Üç bilgi türünü aynı cümlede kesin gerçek gibi birleştirmeyin.

1 **Kökleri keşfet**

README, dizinler, sözleşmeler; stack’i varsayma.

2 **Tek akışı izle**

Ekrandan veriye, iki repo boyunca.

3 **Kanıtı kaydet**

Repo, commit, dosya/sembol, gözlenen davranış.

4 **Çelişki ve kararı ayır**

Onaylı kural ≠ bug ≠ hedef davranış.

ÇIKTI / KAYNAK SICİLİ

- **Gözlendi**

  İncelenen mobil sürüm genel hata gösteriyor.

- **Kararlaştırıldı**

  PM + domain sahibi belirsiz sonucu ayrı gösterme kararını aldı.

- **Bilinmiyor**

  Canlı flag değeri ve eski istemcinin davranışı henüz doğrulanmadı.

Kodu okumak canlı davranışı, test dosyasını okumak da geçen testi kanıtlamaz.

Bağlam sohbetten sonra da yaşamalı.

Context ortak arka planı; rules ilgili belgeyi yönlendirir. İnceleme ve güncelleme sahibi atayın.

openspec/config.yaml · kısa ortak yönlendirme

```
schema: spec-driven
context: |
  Ürün ve domain: ...
  Önce docs/context/README.md oku.
  İlgili kaynakları sürümleriyle belirt.
  Çelişkiyi ve açık kararı görünür kıl.
rules:
  specs:
    - Kritik hata/toparlanma yollarını yaz.
  design:
    - Eski mobil istemci uyumunu değerlendir.
```

docs/context/README.md

Ürün sınırı + repo haritası + okuma rotası

sources.md

Kaynak + sürüm + ortam + güncellik

domain.md

Terimler + kurallar + karar sahipleri

flow.md

Tek akış + kritik dallar + kanıt

Bu docs/context/ düzeni eğitim önerisidir. Dolu örnek şablonlar başlangıç paketinde.

`context` yönergeye eklenir. Bir path yazmak dosyanın tamamını otomatik yüklemez; okuma adımını açık belirtin.

UYGULAMA 08 / 10 · BİLGİSAYARDAN İLK İŞE

Kendi domain'inle ilk incelemeyi başlat.

Alanları değiştir; mesaj anında domain'ine uyarlansın. Bu adım ürün kuralı icat etmez.

Ürün / domain

Sipariş yönetimi

İncelenecek tek akış

Paket değişikliği sonı

Mobil checkout yolu

/path/to/mobile-app

Backend checkout yolu

/path/to/backend

Planlama root'u

/path/to/product-plans

Kapsam dışı

Fiyat, kampanya, uygunluk politikasını değiştirm

```
CODEX SOHBETİ · İNCELEME

$openspec-explore
Ürün/domain: Sipariş yönetimi
Akış: Paket değişikliği sonrası yanıt kaybı
Root: /path/to/product-plans
Mobil: /path/to/mobile-app · commit: <belirt>
Backend: /path/to/backend · commit: <belirt>
Kapsam dışı: Fiyat, kampanya, uygunluk politikasını değiştirmek

Önce erişimi doğrula. Varsa docs/context/README.md ve sources.md oku.
Tek akışı ekran → API → domain → veri → test boyunca izle.
Her bulguya kaynak ve sürüm ekle. Gözlem, varsayım, çelişki ve açık kararları
ayır.
Canlı flag/deploy bilinmiyorsa belirt. Kod değiştirme.
Çıktı: kaynak sicili, akış, kurallar ve karar sahipleri.
```

Yolları gerçek checkout'larla değiştir. Asistan önce erişimi doğrulasın; sonra kaynaklı akış haritasını çıkarsın.

UYGULAMA 09 / 10 · BİLGİSAYARDAN İLK İŞE

İlk iş: araştır → karar → plan → kanıt.

PM'in işi yalnız komut çalıştırmak değil; kaynak, karar ve kabulün tutarlılığını yönetmek.

01 / AI + PM

`$openspec-explore`

Kaynaklı mevcut durum

Akış, kurallar, çelişkiler ve açık karar sahipleri.

02 / PM + DOMAIN SAHİBİ

`Ürün kararlarını netleştir`

Hedef ve sınır

Bir kullanıcı sonucu; başarı sinyali ve kapsam dışı.

03 / AI + EKİP REVIEW

`$openspec-propose <iş>`

Dört belgeyi incele

Proposal, delta specs, design, tasks. Aynı niyet mi?

04 / İLGİLİ COMPONENT

`$openspec-apply-change <iş>`

Uygula ve doğrula

Gerçek test sonucu, ürün kabulü, sync ve archive.

> Review için durma kuralı ekibin çalışma anlaşmasıdır; OpenSpec otomatik PM onay kapısı oluşturmaz.

Plan değişince `$openspec-update-change`; kodu etkiliyorsa yeniden apply. Her çağrıda doğru root'u kontrol et.

UYGULAMA 10 / 10 · BİLGİSAYARDAN İLK İŞE

PM yarın bu işi tek başına başlatabilir mi?

Bu kontrol hazırlık içindir; CLI kurulumu veya ürün kabulü otomatik yapılmaz.

- [x] **CLI sürümü doğrulandı**
  Doğru executable ve Node sürümü.
- [ ] **Root ve skill görülebiliyor**
  Komutun yazacağı yer net.
- [ ] **Mobil + backend erişimi kanıtlandı**
  İlgili checkout/commit ve kaynaklar.
- [ ] **Bağlam ve açık karar sahipleri hazır**
  Gözlem ile ürün kararı ayrıldı.
- [ ] **Tek iş ve kabul senaryoları net**
  Review ve gerçek doğrulama sahibi belli.

`1 / 5 hazırlık noktası tamamlandı.`

İŞ BAŞINDA BAŞVURU

Takılınca doğru yere bak.

Komut bulunamadı
PATH / prefix / aktif Node sürümü

Skill görünmüyor
Doğru tool, generated dosya, yeniden yükleme

AI genel cevap veriyor
Gerçek dosya okuma + kaynak kanıtı

Yanlış proje planı
CWD / root / store / global default

Her PM'e: ortak yöntem + kendi domain paketi + gerçek ilk iş + karar verecek insanlar.

Araç bilgisi: yüksek güven
Bağlam düzeni: ekip önerisi